Conditional Acceptance Calibration for Dynamic Medusa Decoding

ECE 285 Midterm Report

Ada Qi • Alexander Tung • Zhuoting Yang
University of California San Diego
November 2, 2026

Abstract

We investigate whether post-hoc acceptance calibration improves tree selection for frozen Medusa prediction heads. The proposed method estimates each candidate token's acceptance conditional on its parent prefix being accepted, then multiplies these factors to score complete paths. We compare count-based fixed trees and logistic dynamic trees against marginal-accuracy, head-probability, and direct-classification baselines at matched node budgets. The collection, calibration, selection, and benchmarking components are implemented and integrated. A 20-prompt FP32-execution pilot contains 32,806 candidate-node records with recorded greedy-output agreement on every prompt. However, a separate FP16 pilot exposes a numerical mismatch, and complete compatible FP32 traces and broad online validation remain pending. Our current evidence establishes implementation progress and a precision-related correctness issue; it does not yet establish an acceptance or latency advantage for the proposed estimators.

1 Research question and revised scope

Autoregressive generation repeatedly invokes a large language model for individual tokens. Speculative decoding reduces sequential verification work by proposing several tokens together [1]. Medusa uses additional prediction heads to propose future tokens and verifies candidate branches with tree attention [2]. A larger tree explores more continuations but also consumes computation and cache capacity. The practical question is which branches are worth verifying under a fixed budget.

Our research question is unchanged from the proposal: with the backbone and Medusa heads frozen, do parent-conditional acceptance estimates improve candidate selection over marginal-product scores and direct acceptance classifiers at the same tree budget? We use greedy decoding, five Medusa heads, top-10 candidates per head, and the public Vicuna-7B v1.3 Medusa-1 checkpoint. We do not train a new language model or modify the draft heads.

The revised plan adds an explicit numerical-consistency stage before estimator fitting. Tree verification and ordinary greedy generation can disagree in finite precision even with unchanged weights. We therefore treat precision, candidate ranking, label semantics, and trace provenance as experimental variables to control. Complete trace export and validation now precede the final calibration and speed comparisons. HumanEval remains an optional extension; AlpacaEval calibration and MT-Bench evaluation remain the core scope.

2 Mathematical setup and acceptance labels

At a decoding state x, the backbone proposes the next greedy token g_0. Head j proposes an ordered list of K = 10 tokens for offset j after g_0, with j = 1, …, H and H = 5. A draft node v is identified by a rank path (r_1, …, r_d), where ranks are zero-based and d ≤ H. Its ancestors are the shorter prefixes of that path. The draft budget B excludes the backbone root g_0; the tested budgets are 8, 16, 32, 63, and 96.

Let S(v) indicate that the entire candidate prefix through v matches the target model's greedy continuation under the specified execution configuration. Set S(root) = 1. Because accepting a child requires accepting its parent u, the events are nested. For a fixed state and path the label is deterministic; the probabilities below describe variation over collected states and the information retained by our feature map.

$$q(v,x) = P(S(v) = 1 \mid S(u) = 1, x)$$

$$P(S(v) = 1 \mid x) = \prod [q(e,x) : e \text{ on } path(v)] \quad (1)$$

Equation (1) follows from nested acceptance events and the probability chain rule wherever the conditioning event has positive probability. It does not assume independent heads. Our fitted factors remain approximations because counts pool states and logistic regression uses a limited feature representation. In particular, multiplying estimated factors does not by itself guarantee calibrated path probabilities.

The shared TraceSet represents each recorded state by gold_rank [H], topk_prob [H,K], root_prob, prompt_id, and position, with optional topk_token [H,K]. A gold rank of −1 means an observed target token is absent from that head's top-K list. It must never stand for an unobserved future token. For path v, the offline label is one exactly when every rank along the path equals the corresponding gold rank. Candidate ranks come from raw-logit top-K; stored probabilities are gathered from a full-vocabulary FP32 softmax without renormalizing the retained top-K mass.

Offline labels are valid for a policy only when its candidate tokens and verification decisions agree with the trace at the same prefix. Ordinary causal passes, cached generation, and tree attention need not be numerically identical. We check their outputs and candidate ranks rather than assuming equivalence. The existing shared FP16 pilot instead derives labels from its final accepted Medusa sequence and records disagreement with ordinary greedy decoding explicitly. Those labels must not be silently reinterpreted as labels for a different execution mode.

3 Estimators and matched comparisons

The marginal-accuracy baseline estimates a per-head, per-rank frequency and multiplies those frequencies to score a path. The MEDUSA+-style baseline multiplies the current head softmax probabilities [3]. The former yields a fixed tree; the latter changes with the state. Both are useful controls because they retain the same candidate vocabulary and selection budget while omitting explicit conditioning on parent acceptance.

3 Estimators and matched comparisons continued

Our count estimator uses only training examples whose parent prefix is accepted. For head j, previous rank a, and child rank r, let N(j,a,r) count these examples and C(j,a,r) count their accepted children. A dedicated previous-rank bucket represents the root. The implemented smoothing rule is

$$\hat{q}(j,a,r) = [C(j,a,r) + \alpha\,\pi(j,r)] / [N(j,a,r) + \alpha] \quad (2)$$

Here π(j,r) is the acceptance frequency pooled over previous ranks within the accepted-parent examples, not the unconditional marginal head accuracy. Positive α controls shrinkage for sparse cells. This distinction matters: removing previous-rank context still retains conditioning on parent acceptance. The resulting factors are state-independent, so this method produces one fixed tree per budget.

The dynamic conditional estimator fits regularized logistic regression on accepted-parent examples. Features available before verification include depth, capped own and parent ranks, own and parent log confidence, probability relative to the head's top candidate, log path probability, top-K uncertainty, root confidence, and interactions. Features are standardized on training data. The top-K uncertainty feature is a partial entropy statistic, not full-vocabulary entropy. Ada's implementation supports dropping feature groups to measure their contribution.

The direct baseline predicts full-prefix acceptance S(v) rather than a conditional edge event. It uses the same feature map and logistic capacity, but trains on a candidate pool containing both accepted and rejected parents. It is C2T-inspired, not a reproduction of the full C2T system [4]. To reduce the effect of training only on MEDUSA+-selected nodes, the notebook compares a reference pool, a larger reference pool, and an augmented pool containing the direct model's own selected nodes. Selection among these variants uses validation prompts, not the final evaluation set. Direct scores are capped by ancestor scores to enforce monotonicity.

4 Tree construction and its guarantee

All methods use the same ancestor-preserving best-first procedure. Starting from the root, place its children in a priority queue keyed by estimated path score. Repeatedly select the highest-scoring frontier node and add its children until B draft nodes have been selected. Invalid ranks, nonfinite scores, impossible budgets, and child scores above their parent are rejected. Tie handling is deterministic.

Because conditional factors lie in [0,1], their path scores cannot increase along a branch. Every unexpanded descendant is bounded above by its frontier ancestor. Consequently, best-first expansion selects the globally highest-scoring B nodes, with an ancestor-closed ordering available even under ties. This maximizes the sum of the supplied scores for the fixed candidate tree and budget. It does not prove that estimated scores are correct or that this tree minimizes wall-clock latency.

$$E[A(T) \mid x] = 1 + \Sigma\,[P(S(v) = 1 \mid x) : v \in T] \quad (3)$$

Equation (3) follows by linearity of expectation for the number of accepted tokens, including the root, before length or EOS truncation. No independence is needed. If node scores equal the true probabilities, maximizing their sum maximizes expected accepted tokens on these states. With cached feature computation, heap work is approximately O(BK log(BK)); feature evaluation and GPU verification add separate costs.

5 Implementation and preliminary evidence

The shared pipeline now includes trace collection, five scorers, fixed and dynamic tree policies, offline comparison, and separate correctness and latency runners. One canonical feature implementation serves the notebook and decoding pipeline. Fitted logistic models carry a feature-schema identifier because Ada's feature ordering differs from the earlier skeleton. Trace/model execution contracts reject unknown or incompatible precision settings instead of silently mixing artifacts [6].

Evidence	Observed result	Interpretation
FP16 shared pilot	20 prompts; 19 exact greedy matches	One numerical divergence remains
FP32 node pilot	20 prompts; 20 exact greedy matches	Agreement for this pilot and policy
FP32 node records	32,806 total; 8,950 conditional records	Selected-node data, not full TraceSet
FP16 trace replay	498 complete rows; 547 recorded rounds	Recorded default-tree acceptance agrees
CPU integration	69 tests passed; 2 skipped	Software checks, not GPU validation

The FP16 pilot uses NF4 double-quantized weights with FP16 computation and prediction heads. For prompt 24, "What breed dog is smallest?", the first output difference occurs at generated token 32. The competing logits differ by 0.015625, but their ordering reverses between the two execution paths. Making only the language-model prediction head FP32 did not resolve this case. Promoting execution to FP32 while preserving packed 4-bit weights did [7].

The candidate FP32 configuration initially loads and quantizes in FP16, preserves packed NF4 parameters, promotes other floating parameters and buffers, sets 4-bit linear computation to FP32, and uses FP32 prediction heads with an input-casting hook for the language-model head. Our shared implementation additionally disables TF32 explicitly. Weight storage and computation precision are therefore distinct. FP32 execution is our provisional final configuration; its broader validation is still required. The historical FP16 mode remains available and is still the configuration default unless explicitly overridden.

The complete FP32 pilot records 20 exact token-array agreements and 680 accepted-prefix records [7]. Separately, CPU reconstruction of the FP16 NPZ reproduces all six data arrays exactly from its JSON. All 547 recorded default-tree rounds agree with replayed acceptance counts; 49 incomplete-horizon rounds are omitted from the 498-row NPZ rather than labeled negative [8]. These checks establish consistency with the recorded Medusa sequence, not universal agreement with ordinary greedy decoding or arbitrary dynamic trees.

All integrated calibration-notebook code cells ran on a reduced synthetic dataset of 400 calibration and 160 evaluation steps. Both notebook and CLI exports completed five methods at five budgets. The two skipped regression tests require unavailable Medusa/FastChat dependencies. Synthetic fitting and these tests validate software behavior only. We have not yet measured a real-data advantage for conditional calibration or an end-to-end GPU speedup.

6 Experimental design and remaining work

The collected manifest specifies 200 AlpacaEval training prompts, 50 validation prompts, and a 20-prompt pilot drawn only from training. Full compatible FP32 training/validation traces and separate MT-Bench evaluation traces remain outstanding at this reporting checkpoint. We will validate a small shared-collector export first, then collect the agreed splits. Real fitting requires explicit, disjoint manifest IDs that exactly cover the calibration trace IDs. Original prompt text must also be checked for cross-dataset overlap; integer IDs alone are insufficient.

The GPU correctness stage will compare ordinary cached and uncached greedy outputs, static Medusa verification, and dynamic policies on shared prefixes. It will include known mismatch prompts, varied tasks, and longer continuations across all five budgets. Candidate-token ranks and offline/online acceptance labels must agree under the selected precision contract. A passing 20-prompt default-tree pilot is encouraging but cannot replace this broader stage. Original mismatches and collected traces will be preserved.

We will tune count smoothing and logistic regularization/rank caps on validation prompts, then freeze choices before evaluating on MT-Bench. The primary offline outcome is mean accepted tokens per verification step at matched B. Secondary outcomes include Brier score, expected calibration error, and reliability curves on a common candidate pool. Paired uncertainty estimates resample prompts, because multiple states from the same prompt are correlated. Feature ablations and count-context ablations will distinguish calibration effects from feature or pool changes.

Online timing will use the same model, precision, prompts, token limits, and hardware for all policies, with ordinary greedy decoding as the speed reference. Warmups, randomized policy order, synchronization, and separate tree-building profiles will help isolate overhead. Correctness observers and diagnostic passes will not run inside latency measurements. We will report tokens per second and memory separately from trace acceptance. More accepted tokens per round is not sufficient to claim faster generation.

7 Contribution and limitations

The contribution is a controlled study of post-hoc calibration for frozen, non-autoregressive Medusa heads. Conditional acceptance and adaptive trees are not new: TreeSpark already studies calibrated parent-conditioned acceptance in another setting [5]. Our comparison isolates whether this modeling choice helps when draft predictions, candidate sets, tree construction, and budgets are held fixed. The count model supplies an interpretable low-cost intervention; logistic models can adapt branch allocation to state-dependent confidence without changing the backbone.

Important limitations remain. Conditioning produces fewer deep-path examples and sparse rank combinations. Default-tree visited states may not represent states visited by another policy. Multiplying imperfect factors can compound error; monotone clipping changes the direct baseline's scores; and finite-precision changes may alter the underlying target trajectory. FP32 may increase memory and reduce throughput, so fair comparisons must use the same precision rather than crediting calibration for a precision change. We will retain null or negative results if improved calibration does not yield better acceptance or if tree overhead erases the gain.

Work is organized around trace collection and precision validation (Zhuoting), calibration and ablations (Ada), and tree selection and benchmarking (Alexander). The next milestone is a validated real-data handoff, followed by frozen-model evaluation and GPU timing. The midterm deliverable establishes the method, integrated implementation, and pilot evidence; the final empirical research question remains open.

References

[1] Y. Leviathan, M. Kalman, and Y. Matias. Fast Inference from Transformers via Speculative Decoding. ICML, 2023. https://proceedings.mlr.press/v202/leviathan23a.html

[2] T. Cai et al. Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads. ICML, 2024. https://arxiv.org/abs/2401.10774

[3] Z. Zhang. Acceleration Multiple Heads Decoding for LLM via Dynamic Tree Attention. arXiv:2502.05947, 2025. https://arxiv.org/abs/2502.05947

[4] F. Huo, J. Tan, K. Zhang, X. Cai, and S. Sun. C2T: A Classifier-Based Tree Construction Method in Speculative Decoding. arXiv:2502.13652, 2025. https://arxiv.org/abs/2502.13652

[5] H. Zhou et al. TreeSpark: Calibrated, Load-Adaptive Draft Trees for Semi-Autoregressive Speculative Decoding. arXiv:2609.22098, 2026. https://arxiv.org/abs/2609.22098

[6] A. Qi, A. Tung, and Z. Yang. ECE 285 Final Project repository. Integrated code at merge commit cf8dba5afe975f62e9af52bd75404b5aaaf0f03b; ADA_HANDOFF.md and PRECISION.md document CPU validation and remaining checks. https://github.com/alextung2/ECE_285_Final_Project/pull/2

[7] Project pilot evidence. pilot_node_traces_v2_fp32/summary_first_20.json, per-prompt token arrays, and prompt_24_precision_test.json at commit 66d623217dfe218619b86a721f05ac5dec71c72e. https://github.com/alextung2/ECE_285_Final_Project/tree/66d623217dfe218619b86a721f05ac5dec71c72e/pilot_node_traces_v2_fp32

[8] Project FP16 shared trace evidence. shared_traces_fp16_v1/pilot/calib.npz, per-prompt JSON, collection_settings.json, and summary_first_20.json at commit 66d623217dfe218619b86a721f05ac5dec71c72e. https://github.com/alextung2/ECE_285_Final_Project/tree/66d623217dfe218619b86a721f05ac5dec71c72e/shared_traces_fp16_v1/pilot